Introduction

This task continues the payment calculator by accepting values from the user, converting them to numeric types, organising the calculation into functions, creating a reusable module, and planning the algorithm before coding.

TASK 5 — Read Input and Convert Types

The input() function reads text from the keyboard. Since input() returns a string, float() is used to convert the milk quantity and rate into numbers before multiplication.

```
milk_quantity = float(input("Enter milk quantity in litres: "))
rate_per_litre = float(input("Enter rate per litre: "))

total_payment = milk_quantity * rate_per_litre
print("Total payment: Rs.", round(total_payment, 2))
```

Example: entering 12.5 for quantity and 38 for rate gives a total payment of ■475.00. If the user enters text that is not numeric, float() raises a ValueError; a more robust version can handle this error.

```
try:
    milk_quantity = float(input("Enter milk quantity in litres: "))
    rate_per_litre = float(input("Enter rate per litre: "))
    if milk_quantity < 0 or rate_per_litre < 0:
        print("Quantity and rate must not be negative.")
    else:
        print("Total payment: Rs.", round(milk_quantity * rate_per_litre, 2))
except ValueError:
    print("Please enter valid numbers.")
```

TASK 6 — Organise the Logic into Functions

A function is a reusable block of code that performs a specific task. Separating calculation from display makes the program easier to read, test and maintain.

```
def calculate_payment(milk_quantity, rate_per_litre):
    return milk_quantity * rate_per_litre

quantity = 12.5
rate = 38.0
payment = calculate_payment(quantity, rate)
print("Total payment: Rs.", round(payment, 2))
```

The function receives two arguments and returns their product. It does not depend on values typed directly inside the function.

TASK 7 — Build a Module

A module is a Python file containing reusable code. Save the calculation function in a file named payment_calculator.py, then import it into another Python file.

File 1: payment_calculator.py

```
def calculate_payment(milk_quantity, rate_per_litre):
    if milk_quantity < 0 or rate_per_litre < 0:
        raise ValueError("Quantity and rate must not be negative")
    return milk_quantity * rate_per_litre
```

File 2: main.py

```
from payment_calculator import calculate_payment

quantity = float(input("Enter milk quantity in litres: "))
rate = float(input("Enter rate per litre: "))

try:
    payment = calculate_payment(quantity, rate)
    print("Total payment: Rs.", round(payment, 2))
```

```
except ValueError as error:
    print(error)
```

Both files should be saved in the same folder. The import statement allows main.py to use the function from the module.

TASK 8 — Plan the Algorithm Before Coding

Algorithm

1. Start the program.
2. Read the milk quantity from the user.
3. Read the payment rate per litre.
4. Convert both inputs to floating-point numbers.
5. Check that neither value is negative.
6. Calculate payment = quantity × rate.
7. Display the payment rounded to two decimal places.
8. If input is not numeric, display an error message.
9. Stop the program.

Pseudocode

```
BEGIN
    INPUT quantity
    INPUT rate
    IF quantity or rate is not numeric
        DISPLAY input error
    ELSE IF quantity < 0 OR rate < 0
        DISPLAY invalid value message
    ELSE
        payment = quantity * rate
        DISPLAY payment to 2 decimal places
    END IF
END
```

Conclusion

The calculator now accepts user input, converts values to the correct types, uses a function for the calculation, and stores reusable logic in a separate module. Planning the algorithm first makes the program easier to implement and helps identify validation requirements before testing.